Network and System Programming

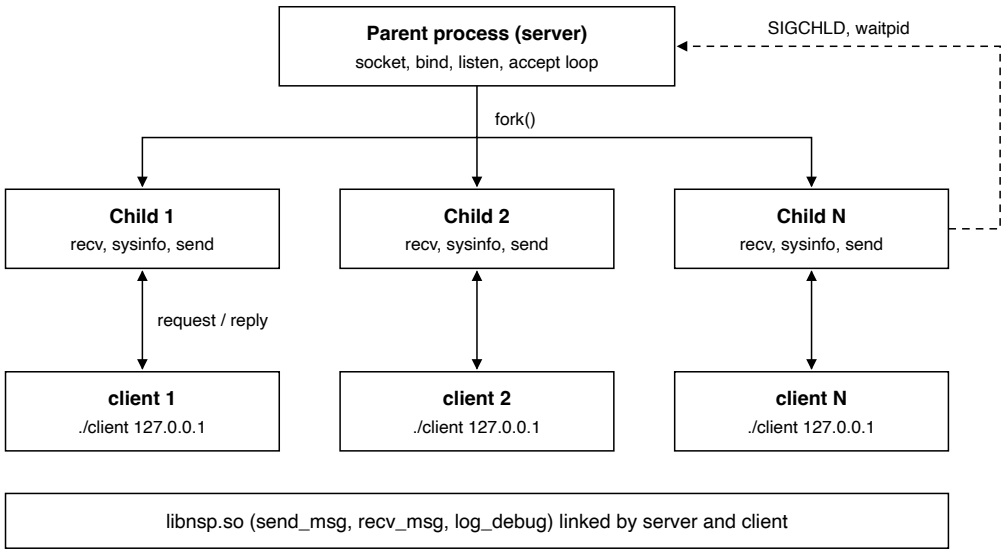
# Midterm Project Report

fork-based Client–Server System Information Service

**Source code** https://github.com/Ansenchen123/network-system-programming

## 1. System Design

### 1.1 Architecture




### 1.2 Components

| Component | Files | Responsibility |
|---|---|---|
| Shared library | `nsp.h`, `nsp.c` → `libnsp.so` | `send_msg`, `recv_msg`, `log_debug` |
| Server | `server.c` | Port 9000; one `fork()` per connection; system information; 3 robustness mechanisms |
| Client | `client.c` | Sends `sysinfo`, prints the reply |
| Concurrency test | `test10.sh` | 10 clients at once |
| Build | `Makefile` | Library and both programs |

### 1.3 Message Protocol

| Field | Size | Content |
|---|---|---|
| Length | 4 bytes | Data length, network byte order |
| Data | ≤ 4095 bytes | Request: `sysinfo`. Reply: system information text |

### 1.4 System Information Returned

| Item | Source |
|---|---|
| Host name, kernel version | `uname()` |
| Uptime, load averages, free/total memory, process count | `sysinfo()` |
| PID of the serving child | `getpid()` |

## 2. Requirements Checklist

| Requirement | Implementation | Proof |
|---|---|---|
| `fork()` per connection, no threads | `fork()` after `accept` | Figure 3 |
| ≥ 10 concurrent clients | `test10.sh` | Figure 3 |
| Packet capture and analysis | `tcpdump -i lo`, `tshark` | Section 5, Figure 6 |
| Compile-time debug log control | `make DEBUG=1` → `-DDEBUG` | Figures 4, 5 |
| Runtime debug log control | `DEBUG_LOG=1` | Figures 4, 5 |
| Shared utilities in a `.so` | `libnsp.so` | Figure 1 |
| Client requests system information | `sysinfo` request | Figure 2 |
| Robustness, ≥ 3 with/without comparisons | 3 mechanisms; `./server norobust` disables them | Section 4, Figures 7–9 |
| Makefile or CMake | `Makefile` | Figure 1 |

# 3. Key Source Code

### 3.1 The fork loop (`main` in `server.c`)

```
while (1) {
    peer_len = sizeof(peer);
    client_fd = accept(listen_fd, (struct sockaddr *)&peer, &peer_len);
    if (client_fd < 0) {
        perror("accept");
        continue;
    }
    pid = fork();
    if (pid == 0) {
        close(listen_fd);
        printf("[pid %d] serving %s:%d\n", getpid(), inet_ntoa(peer.sin_addr), ntohs(pee
r.sin_port));
        handle_client(client_fd, robust);
        close(client_fd);
        printf("[pid %d] done\n", getpid());
        exit(0);
    }
    close(client_fd);
}
```

The child serves the client; the parent returns to `accept`.

### 3.2 Reaping children (`reap_children`)

```
static void reap_children(int sig)
{
    while (waitpid(-1, NULL, WNOHANG) > 0) {
    }
}
```

Reaps every exited child; `WNOHANG` never blocks.

### 3.3 Message receive (`nsp.c`)

```
int recv_msg(int fd, char *buf, int size)
{
    uint32_t net_len;
    int len;

    if (read_all(fd, (char *)&net_len, 4) < 0) {
        return -1;
    }
    len = ntohl(net_len);
    if (len < 0 || len >= size) {
        return -1;
    }
    if (read_all(fd, buf, len) < 0) {
        return -1;
    }
    buf[len] = '\0';
    return len;
}
```

Reads the 4-byte length, checks it against the buffer, then reads exactly that many bytes.

### 3.4 Two-level debug log (`log_debug`)

```
void log_debug(const char *fmt, ...)
{
#ifdef DEBUG
    va_list args;
    char *env = getenv("DEBUG_LOG");

    if (env == NULL || strcmp(env, "1") != 0) {
        return;
    }
    fprintf(stderr, "[DEBUG pid %d] ", getpid());
    va_start(args, fmt);
    vfprintf(stderr, fmt, args);
    va_end(args);
    fprintf(stderr, "\n");
#endif
}
```

Compile time: `#ifdef DEBUG`. Run time: `DEBUG_LOG=1`.

# 4. Robustness Mechanisms

| # | Abnormal condition | Without (`./server norobust`) | With (`./server`) | Mechanism |
|---|---|---|---|---|
| 1 | Exited children are not reaped | 3 clients → 3 `Z <defunct>` | No zombies | `SIGCHLD` handler + `waitpid(WNOHANG)` |
| 2 | Client connects, sends nothing | Child blocks in `recv` forever | Disconnects after 5 s | `SO_RCVTIMEO` |
| 3 | Server restart | `bind: Address already in use` for ~60 s | Restarts immediately | `SO_REUSEADDR` |

### 4.1 Mechanism 1: reaping children

```
# Without: ./server norobust, 3 clients
    PID    PPID STAT CMD
   2059    2056 Z    [server] <defunct>
   2061    2056 Z    [server] <defunct>
   2063    2056 Z    [server] <defunct>

# With: ./server, 3 clients
    PID    PPID STAT CMD
```

### 4.2 Mechanism 2: request timeout

```
# Without: ./client 127.0.0.1 8, child still waiting at second 6
    PID STAT CMD
   2197 S    ./server norobust

# With: server disconnects at second 5
[pid 1932] serving 127.0.0.1:38712
[pid 1932] no valid request (timeout or disconnected)
[pid 1932] done
# Client:
send failed (server closed the connection)
```

### 4.3 Mechanism 3: port reuse

```
# Without: restart after serving a client
TIME-WAIT 0      0          127.0.0.1:9000      127.0.0.1:58980
bind: Address already in use

# With: restart after serving a client
server pid 1948 listening on port 9000 (robust=1)
```

## 5. Packet Capture and Analysis

```
sudo tcpdump -i lo -nn -w midterm.pcap tcp port 9000
./client 127.0.0.1
tshark -r midterm.pcap
```

Client port 40944, server port 9000.

| # | Direction | Flags | Data length | Meaning |
|---|---|---|---|---|
| 1 | client → server | SYN | 0 | Handshake (`connect`) |
| 2 | server → client | SYN, ACK | 0 | Handshake |
| 3 | client → server | ACK | 0 | Handshake done; `accept` returns |
| 4 | client → server | PSH, ACK | 4 | Request length: `00 00 00 07` |
| 5 | server → client | ACK | 0 | |
| 6 | client → server | PSH, ACK | 7 | Request data: `sysinfo` |
| 7 | server → client | ACK | 0 | |
| 8 | server → client | PSH, ACK | 4 | Reply length: 179 |
| 9 | client → server | ACK | 0 | |
| 10 | server → client | PSH, ACK | 179 | Reply data: system information |
| 11 | server → client | FIN, ACK | 0 | Server closes first → `TIME_WAIT` on the server |
| 12 | client → server | ACK | 0 | |
| 13 | client → server | FIN, ACK | 0 | Client closes |
| 14 | server → client | ACK | 0 | Connection closed |

## 6. Execution Screenshots

Top pane: server. Bottom pane: client.

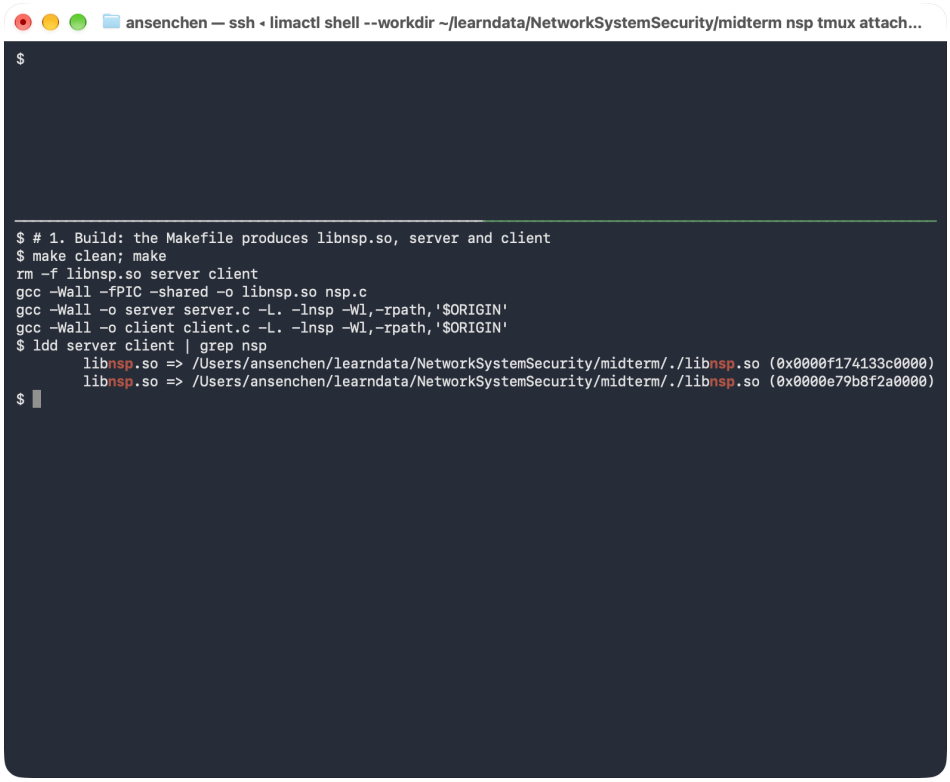



**Figure 1.** Build; both programs link `libnsp.so`.

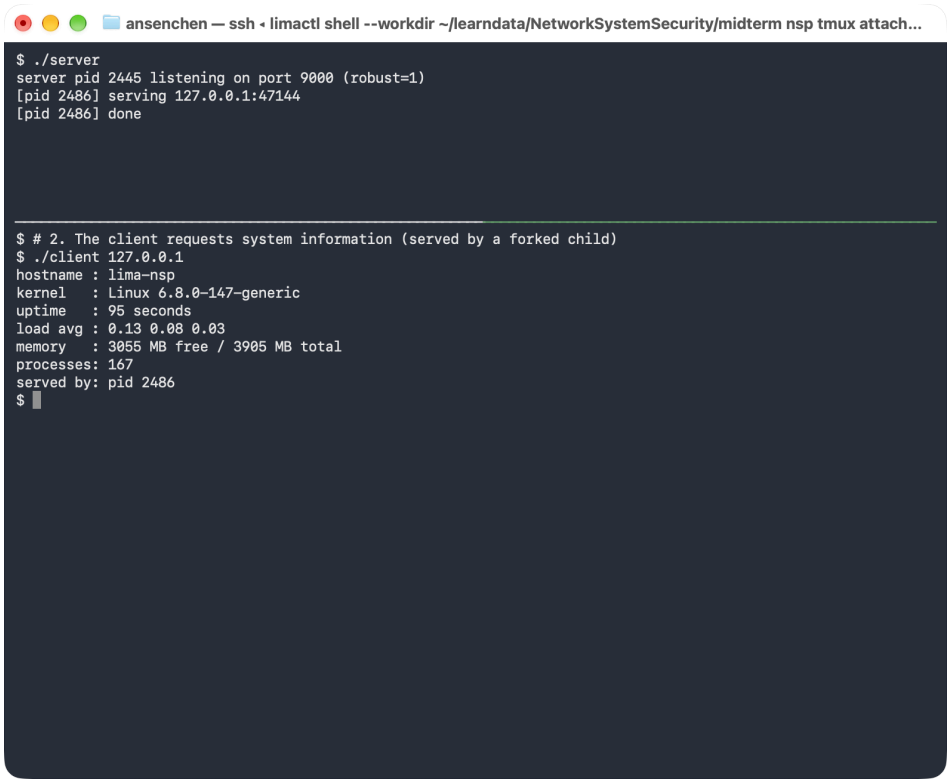



**Figure 2.** Client receives system information.

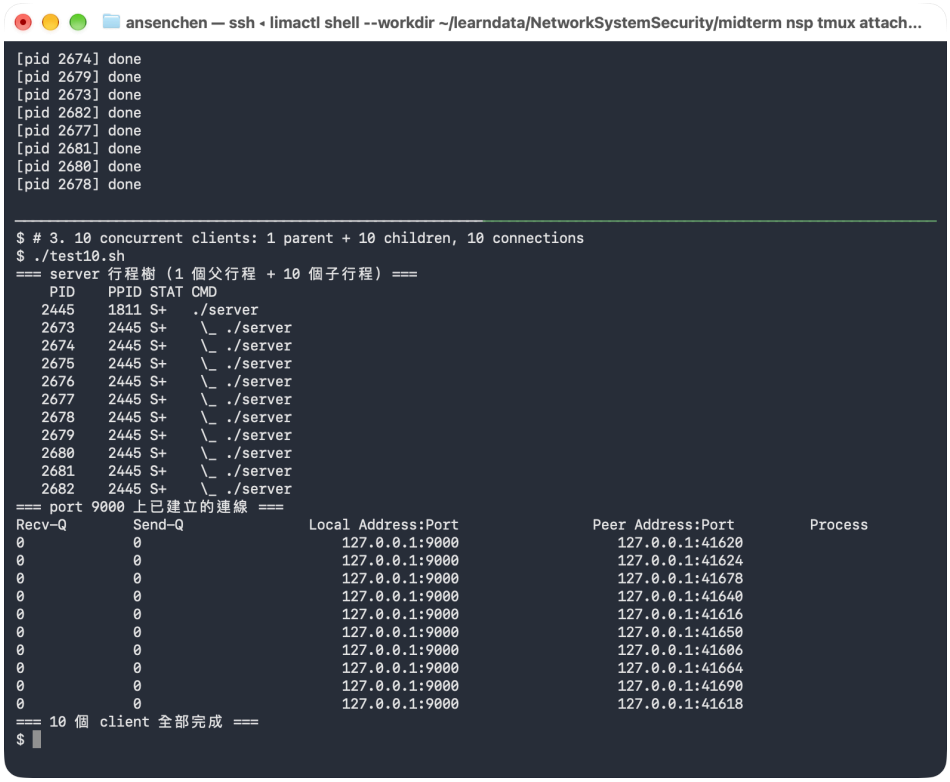



**Figure 3.** 10 concurrent clients: 1 parent + 10 children, 10 connections.

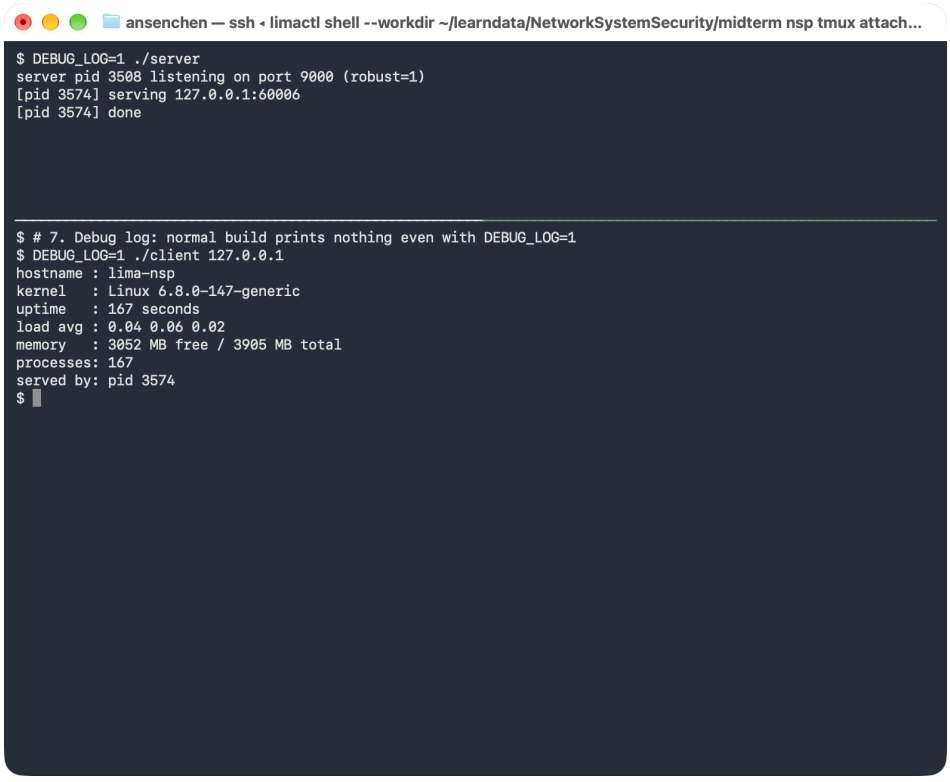



**Figure 4.** Normal build + `DEBUG_LOG=1`: no debug output.

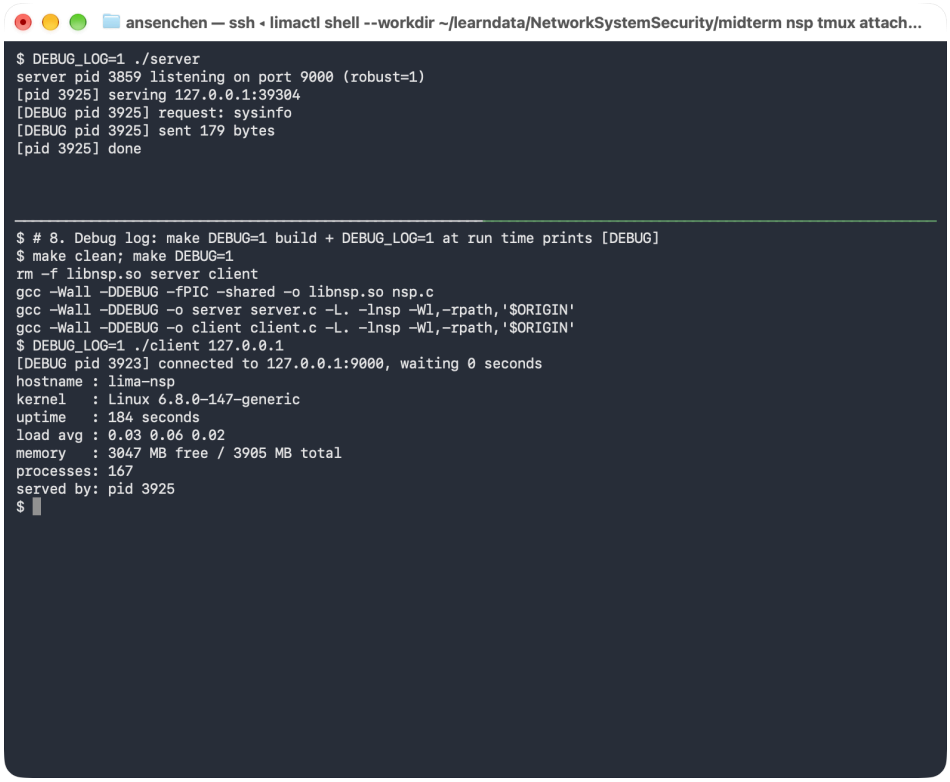



**Figure 5.** `make DEBUG=1` + `DEBUG_LOG=1`: debug output.

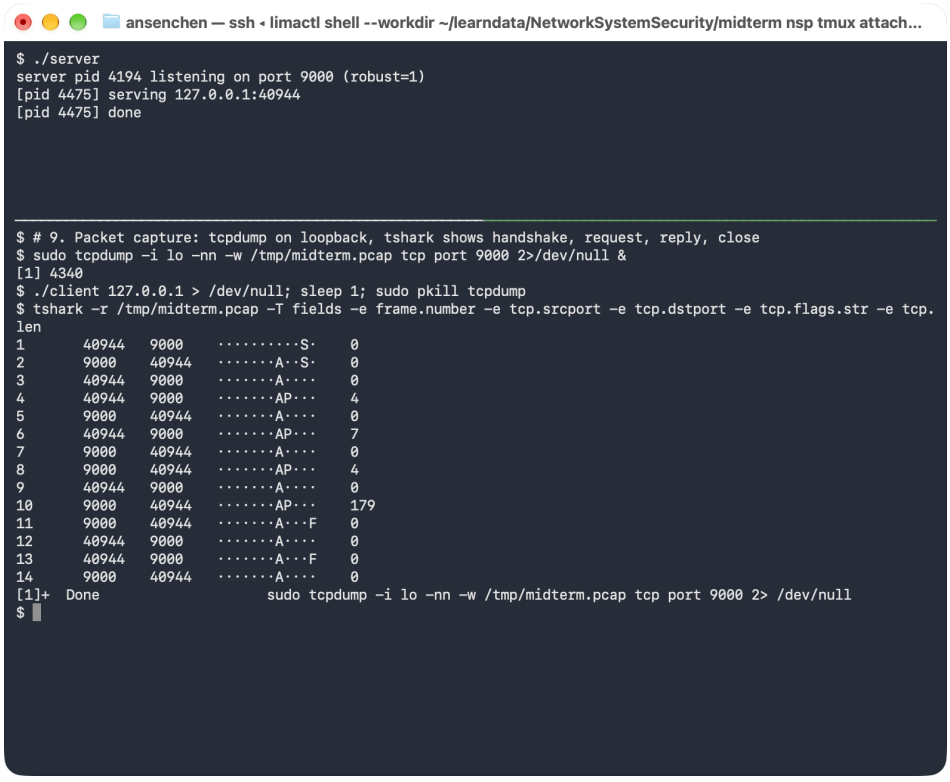



**Figure 6.** Packet capture: packet number, source port, destination port, flags, data length.

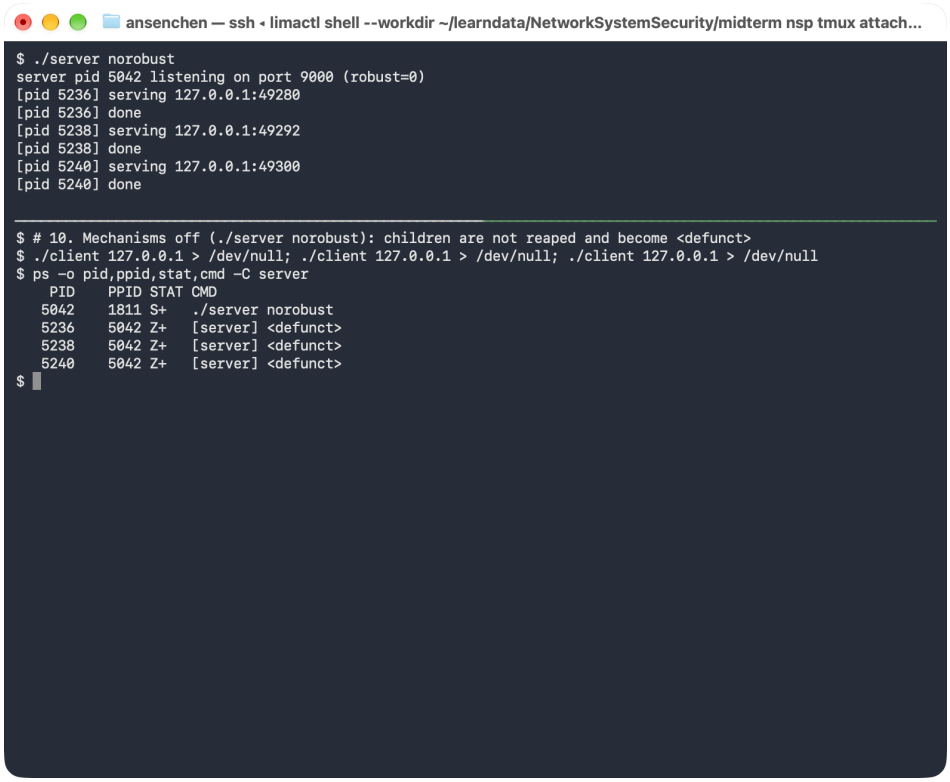



**Figure 7a.** Mechanism 1 off: zombie processes.

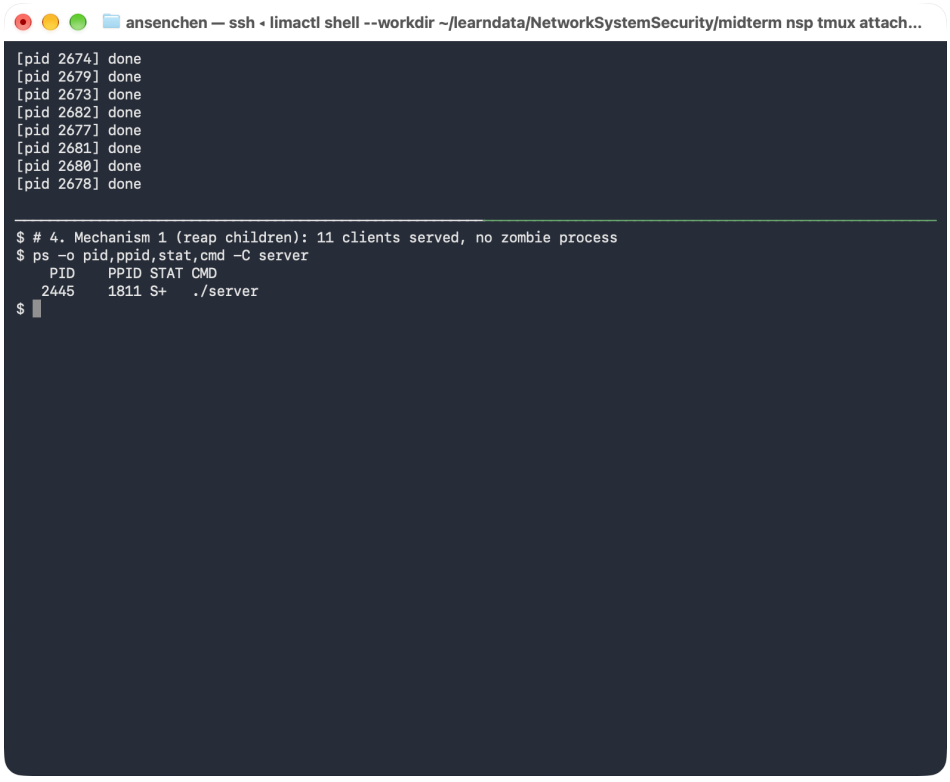



**Figure 7b.** Mechanism 1 on: no zombie processes.

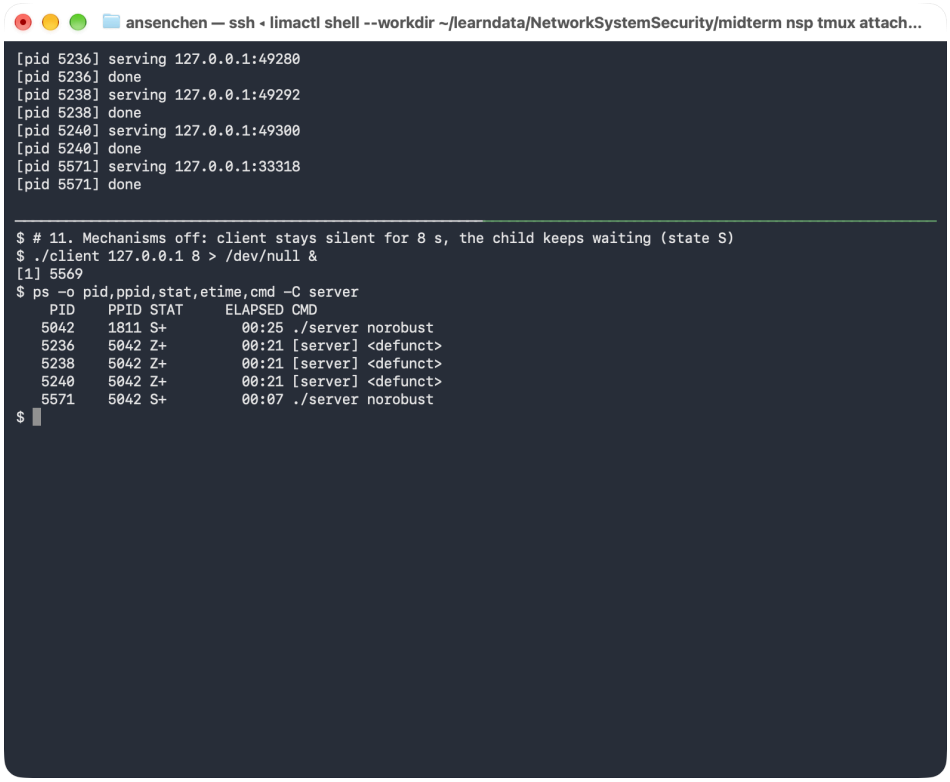



**Figure 8a.** Mechanism 2 off: the child keeps waiting.

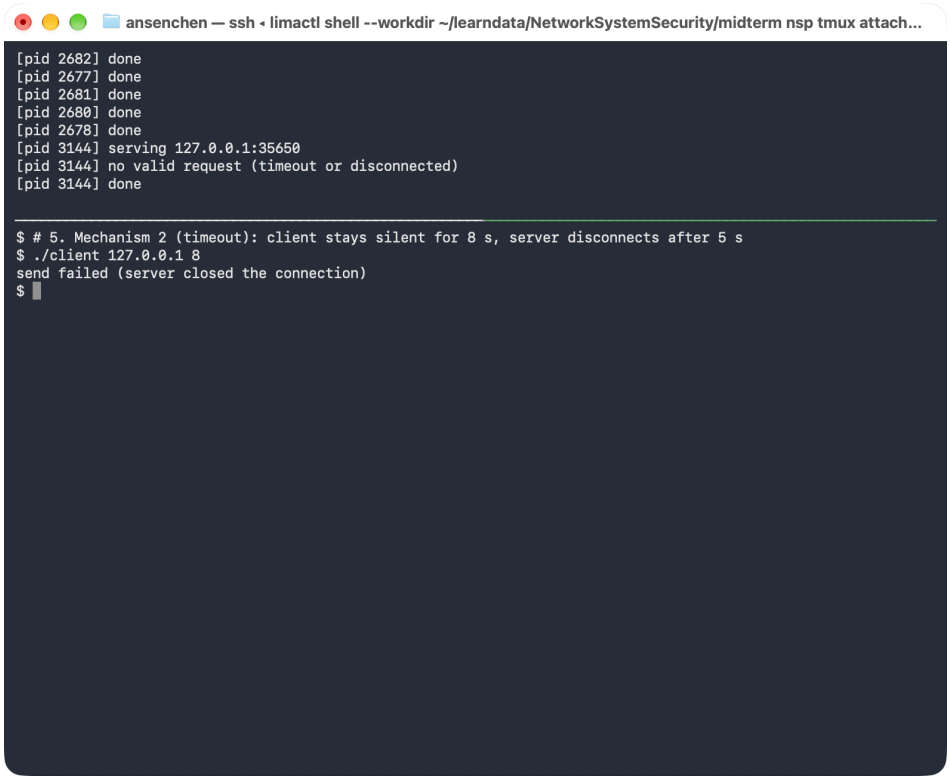



**Figure 8b.** Mechanism 2 on: 5-second timeout.

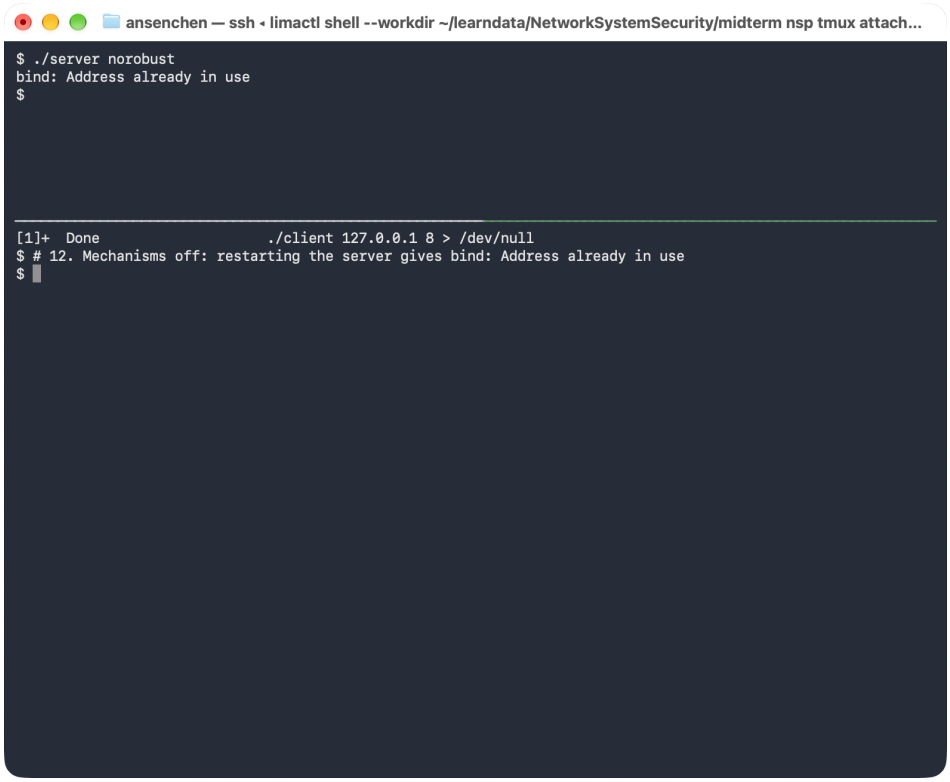



**Figure 9a.** Mechanism 3 off: `bind` fails.

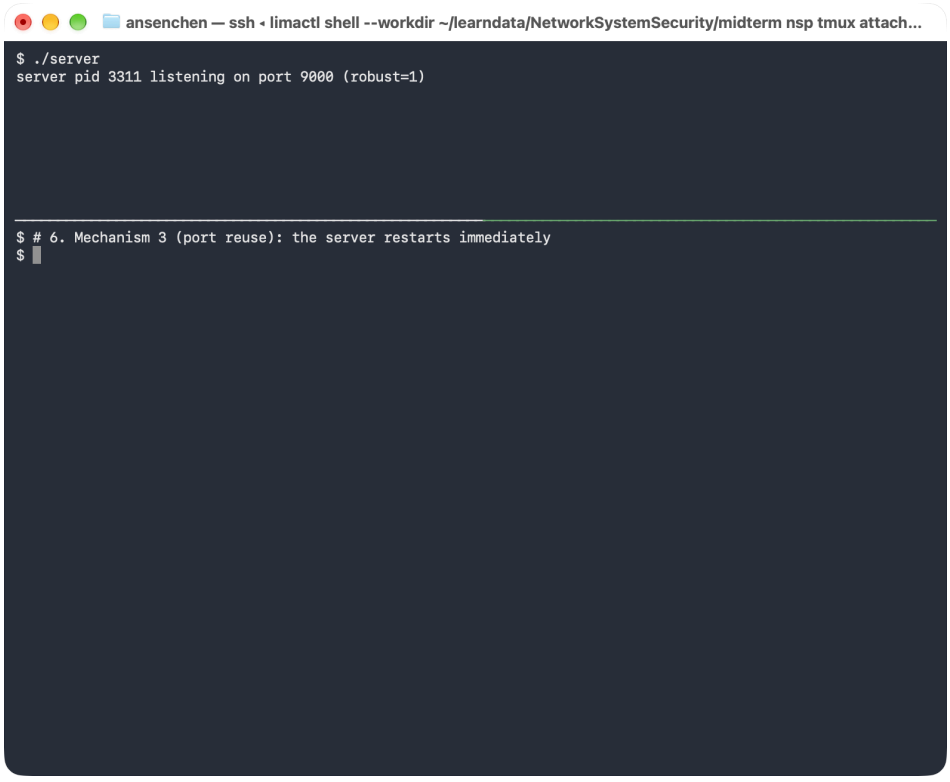



**Figure 9b.** Mechanism 3 on: immediate restart.

## 7. Build and Run

Ubuntu 24.04 (arm64), gcc.

| Command | Purpose |
|---|---|
| `make` | Normal build; debug log not compiled in |
| `make DEBUG=1` | Compile the debug log in (run `make clean` before switching) |
| `make clean` | Remove build outputs |
| `./server` | Start the server with the robustness mechanisms on |
| `./server norobust` | Start the server with the mechanisms off |
| `./client <IP> [wait seconds]` | Connect and get system information; the wait creates the concurrency and timeout scenarios |
| `./test10.sh` | Concurrency test with 10 clients |
| `DEBUG_LOG=1 ./server` | Turn the debug log on at run time |

# 8. Complete Source Code

Repository: https://github.com/Ansenchen123/network-system-programming

### nsp.h

```
/* nsp.h：client 與 server 共用的常數與函式宣告，函式實作在 nsp.c（編成 libnsp.so） */

#define PORT 9000      /* server 監聽的埠號 */
#define MAX_MSG 4096   /* 一則訊息的最大長度（位元組） */

int send_msg(int fd, const char *data, int len);
int recv_msg(int fd, char *buf, int size);
void log_debug(const char *fmt, ...);
```

### nsp.c

```
/* nsp.c：client 與 server 共用的工具函式，編譯成動態函式庫 libnsp.so */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <stdint.h>
#include <unistd.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include "nsp.h"

/*
 * write_all：把 buf 的 len 個位元組全部送出。
 *   send() 一次可能只送出一部分，所以用迴圈送到全部送完為止。
 *   MSG_NOSIGNAL：對方已斷線時回傳錯誤，而不是收到 SIGPIPE 讓整個行程被殺掉。
 * 輸入：fd 已連線的 socket；buf 要送的資料；len 要送的位元組數
 * 輸出：全部送完回傳 0；失敗回傳 -1
 */
static int write_all(int fd, const char *buf, int len)
{
    int done = 0;
    int n;

    while (done < len) {
        n = send(fd, buf + done, len - done, MSG_NOSIGNAL);
        if (n <= 0) {
            return -1;
        }
        done = done + n;
    }
    return 0;
}

/*
 * read_all：從 fd 讀滿 len 個位元組到 buf。
 *   recv() 一次可能只讀到一部分，所以用迴圈讀到讀滿為止。
 * 輸入：fd 已連線的 socket；buf 存放資料的緩衝區；len 要讀的位元組數
 * 輸出：讀滿回傳 0；對方斷線、逾時或出錯回傳 -1
 */
static int read_all(int fd, char *buf, int len)
{
    int done = 0;
```

```
    int n;

    while (done < len) {
        n = recv(fd, buf + done, len - done, 0);
        if (n <= 0) {
            return -1;
        }
        done = done + n;
    }
    return 0;
}

/*
 * send_msg：送出一則訊息。
 *   訊息格式：[4 位元組資料長度 (網路位元組順序)][資料]
 * 輸入：fd 已連線的 socket；data 要送的資料；len 資料長度
 * 輸出：成功回傳 0；失敗回傳 -1
 */
int send_msg(int fd, const char *data, int len)
{
    uint32_t net_len = htonl(len);

    if (write_all(fd, (const char *)&net_len, 4) < 0) {
        return -1;
    }
    return write_all(fd, data, len);
}

/*
 * recv_msg：收一則 send_msg 送來的訊息，並在資料結尾補 '\0'，方便當字串使用。
 * 輸入：fd 已連線的 socket；buf 存放資料的緩衝區；size 緩衝區大小
 * 輸出：成功回傳資料長度；對方斷線、逾時、或資料長度超過緩衝區回傳 -1
 */
int recv_msg(int fd, char *buf, int size)
{
    uint32_t net_len;
    int len;

    if (read_all(fd, (char *)&net_len, 4) < 0) {
        return -1;
    }
    len = ntohl(net_len);
    if (len < 0 || len >= size) {
        return -1;
    }
    if (read_all(fd, buf, len) < 0) {
        return -1;
    }
    buf[len] = '\0';
    return len;
}

/*
 * log_debug：把除錯訊息印到 stderr，用法和 printf 相同。
 *   編譯期開關：用 make DEBUG=1 編譯時函式才有內容，否則是空函式，什麼都不印。
 *   執行期開關：執行時設定環境變數 DEBUG_LOG=1 才會真的印出。
 * 輸入：fmt 格式字串；... 對應 fmt 的參數
 * 輸出：無
 */
void log_debug(const char *fmt, ...)
{
#ifdef DEBUG
    va_list args;
    char *env = getenv("DEBUG_LOG");
```

```
    if (env == NULL || strcmp(env, "1") != 0) {
        return;
    }
    fprintf(stderr, "[DEBUG pid %d] ", getpid());
    va_start(args, fmt);
    vfprintf(stderr, fmt, args);
    va_end(args);
    fprintf(stderr, "\n");
#endif
}
```

**server.c**

```
/* server.c：每接到一個 client 連線就 fork 一個子行程服務它，回傳本機系統資訊 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/socket.h>
#include <sys/sysinfo.h>
#include <sys/utsname.h>
#include <sys/time.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include "nsp.h"

#define TIMEOUT_SEC 5   /* 防護 2：等待 client 請求的最長秒數 */

/*
 * reap_children：SIGCHLD 的處理函式，子行程結束時由核心呼叫。
 *   用 waitpid 收掉所有已結束的子行程，避免它們變成殭屍行程。
 * 輸入：sig 訊號編號（不使用）
 * 輸出：無
 */
static void reap_children(int sig)
{
    while (waitpid(-1, NULL, WNOHANG) > 0) {
    }
}

/*
 * build_sysinfo：查詢本機系統資訊，寫成一段文字。
 * 輸入：buf 存放文字的緩衝區；size 緩衝區大小
 * 輸出：回傳寫入的文字長度（不含結尾 '\0'）
 */
static int build_sysinfo(char *buf, int size)
{
    struct utsname u;
    struct sysinfo s;

    uname(&u);
    sysinfo(&s);
    return snprintf(buf, size,
                    "hostname : %s\n"
                    "kernel   : %s %s\n"
                    "uptime   : %ld seconds\n"
                    "load avg : %.2f %.2f %.2f\n"
                    "memory   : %lu MB free / %lu MB total\n"
                    "processes: %d\n"
                    "served by: pid %d\n",
```

```
                    u.nodename,
                    u.sysname, u.release,
                    s.uptime,
                    s.loads[0] / 65536.0, s.loads[1] / 65536.0, s.loads[2] / 65536.0,
                    s.freeram * s.mem_unit / 1024 / 1024, s.totalram * s.mem_unit / 1024 / 102
4,
                    s.procs,
                    getpid());
}

/*
 * handle_client：子行程服務一個 client：收請求、回應、結束。
 * 輸入：fd 與 client 連線的 socket；robust 1 表示開啟防護，0 表示關閉
 * 輸出：無
 */
static void handle_client(int fd, int robust)
{
    char req[MAX_MSG];
    char reply[MAX_MSG];
    int len;
    struct timeval tv;

    if (robust == 1) {
        /* 防護 2：client 超過 TIMEOUT_SEC 秒沒送請求，recv 就回傳錯誤，子行程不會永遠卡住 */
        tv.tv_sec = TIMEOUT_SEC;
        tv.tv_usec = 0;
        setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));
    }
    if (recv_msg(fd, req, MAX_MSG) < 0) {
        printf("[pid %d] no valid request (timeout or disconnected)\n", getpid());
        return;
    }
    log_debug("request: %s", req);
    if (strcmp(req, "sysinfo") == 0) {
        len = build_sysinfo(reply, MAX_MSG);
    } else {
        len = snprintf(reply, MAX_MSG, "unknown request: %s\n", req);
    }
    if (send_msg(fd, reply, len) < 0) {
        printf("[pid %d] send failed (client disconnected)\n", getpid());
        return;
    }
    log_debug("sent %d bytes", len);
}

/*
 * main：流程——建立監聽 socket，每接到一個連線就 fork 一個子行程去服務。
 *   用法：./server            開啟防護機制
 *         ./server norobust   關閉防護機制（示範用）
 */
int main(int argc, char *argv[])
{
    int robust = 1;
    int listen_fd;
    int client_fd;
    int opt = 1;
    pid_t pid;
    struct sockaddr_in addr;
    struct sockaddr_in peer;
    socklen_t peer_len;
    struct sigaction sa;

    if (argc > 1 && strcmp(argv[1], "norobust") == 0) {
        robust = 0;
    }
```

```
    /* stdout 改成每行立刻輸出，避免 fork 時子行程複製到還沒印出的內容而重複印 */
    setvbuf(stdout, NULL, _IOLBF, 0);

    if (robust == 1) {
        /* 防護 1：子行程結束時自動回收，避免殭屍行程；SA_RESTART 讓被打斷的 accept 自動重來 */
        memset(&sa, 0, sizeof(sa));
        sa.sa_handler = reap_children;
        sa.sa_flags = SA_RESTART;
        sigaction(SIGCHLD, &sa, NULL);
    }

    listen_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (robust == 1) {
        /* 防護 3：允許重用還在 TIME_WAIT 狀態的埠，server 重啟時 bind 不會失敗 */
        setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    }
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(PORT);
    if (bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("bind");
        return 1;
    }
    listen(listen_fd, 16);
    printf("server pid %d listening on port %d (robust=%d)\n", getpid(), PORT, robust);

    while (1) {
        peer_len = sizeof(peer);
        client_fd = accept(listen_fd, (struct sockaddr *)&peer, &peer_len);
        if (client_fd < 0) {
            perror("accept");
            continue;
        }
        pid = fork();
        if (pid == 0) {
            close(listen_fd);
            printf("[pid %d] serving %s:%d\n", getpid(), inet_ntoa(peer.sin_addr), ntohs(peer.
sin_port));
            handle_client(client_fd, robust);
            close(client_fd);
            printf("[pid %d] done\n", getpid());
            exit(0);
        }
        close(client_fd);
    }
}
```

## client.c

```
/* client.c：連到 server，請求系統資訊並印出 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include "nsp.h"

/*
 * main：流程──連到 server，等待指定秒數，送出 "sysinfo" 請求，印出回應。
```

```
 *   用法：./client <server IP> [等待秒數]
 *   等待秒數：連上後先等幾秒才送請求，用來製造「多個連線同時存在」與「逾時」的情境。
 */
int main(int argc, char *argv[])
{
    int fd;
    int wait_sec = 0;
    struct sockaddr_in addr;
    char reply[MAX_MSG];

    if (argc < 2) {
        printf("usage: %s <server_ip> [wait_seconds]\n", argv[0]);
        return 1;
    }
    if (argc > 2) {
        wait_sec = atoi(argv[2]);
    }

    fd = socket(AF_INET, SOCK_STREAM, 0);
    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(PORT);
    if (inet_pton(AF_INET, argv[1], &addr.sin_addr) != 1) {
        printf("invalid IP address: %s\n", argv[1]);
        return 1;
    }
    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("connect");
        return 1;
    }
    log_debug("connected to %s:%d, waiting %d seconds", argv[1], PORT, wait_sec);
    sleep(wait_sec);

    if (send_msg(fd, "sysinfo", 7) < 0) {
        printf("send failed (server closed the connection)\n");
        return 1;
    }
    if (recv_msg(fd, reply, MAX_MSG) < 0) {
        printf("no reply (server closed the connection)\n");
        return 1;
    }
    printf("%s", reply);
    close(fd);
    return 0;
}
```

## Makefile

```
# 用法：make             一般編譯（除錯訊息不編進去）
#       make DEBUG=1     把除錯訊息編進去（切換前先 make clean）
#       make clean       刪除編譯產物

CFLAGS = -Wall

ifdef DEBUG
CFLAGS = -Wall -DDEBUG
endif

all: libnsp.so server client

# 共用函式庫：-fPIC 產生可重定位的程式碼，-shared 輸出成動態函式庫
libnsp.so: nsp.c nsp.h
        gcc $(CFLAGS) -fPIC -shared -o libnsp.so nsp.c
```

```
# -L. -lnsp 連結 libnsp.so；-rpath '$$ORIGIN' 讓執行時到執行檔所在的資料夾找 libnsp.so
server: server.c nsp.h libnsp.so
        gcc $(CFLAGS) -o server server.c -L. -lnsp -Wl,-rpath,'$$ORIGIN'

client: client.c nsp.h libnsp.so
        gcc $(CFLAGS) -o client client.c -L. -lnsp -Wl,-rpath,'$$ORIGIN'

clean:
        rm -f libnsp.so server client
```

### test10.sh

```
#!/bin/bash
# 併發測試：同時啟動 10 個 client，每個連上後等 5 秒才送請求。
# 在這 5 秒內列出 server 的行程樹與連線，證明 10 個 client 同時被 10 個子行程服務。
# 用法：先在另一個視窗執行 ./server，再執行 ./test10.sh

for i in 1 2 3 4 5 6 7 8 9 10; do
    ./client 127.0.0.1 5 > /dev/null &
done
sleep 1

echo "=== server 行程樹（1 個父行程 + 10 個子行程）==="
ps -o pid,ppid,stat,cmd --forest -C server

echo "=== port 9000 上已建立的連線 ==="
ss -tn state established '( sport = :9000 )'

wait
echo "=== 10 個 client 全部完成 ==="
```